

Catalog Consolidation

Uma CLI determinística para consolidar ofertas de sellers sem duplicar produtos — com identidade explicável, migração segura e idempotência comprovada.

Requisitos VTEX, arquitetura, dados, testes e defesa técnica

O desafio e a decisão central

Receber itens de múltiplos sellers, persistir no SQLite fornecido e evitar duplicar um produto já existente apesar de pequenas variações.

Critério principal

Entendimento e decisões defensáveis importam mais do que simular escala de produção.

Ambiguidade intencional

O domínio não oferece uma identidade universal; a política precisa ser explícita.

Liberdade técnica

Alterar o schema é permitido quando a mudança preserva dados e explica seu motivo.

Resposta: identidade canônica conservadora + transação SQLite única + constraints reais + evidência reproduzível.

Quatro autoridades, sem confusão

USER	workflow SDD, Harness, OpenAI, custo, entrega	→ requirements IDs → SDD → AC → evidência
ASSESSMENT	comportamento esperado do consolidator	→ requirements IDs → SDD → AC → evidência
PROCESS	uso responsável de IA e avaliação	→ requirements IDs → SDD → AC → evidência
FIXTURE	fatos observados nos snapshots	→ requirements IDs → SDD → AC → evidência

Arquivos JSON/SQLite são evidência da amostra — nunca regra universal do domínio.

O que os dados realmente dizem

269

linhas de entrada

268

relações seller+ID
distintas

975

produtos iniciais

0

vínculos SellerProduct

Schema da entrada

Campos exatos: Id, SellerName, Name, Brand, Category. Brand é nulo em 3 linhas.

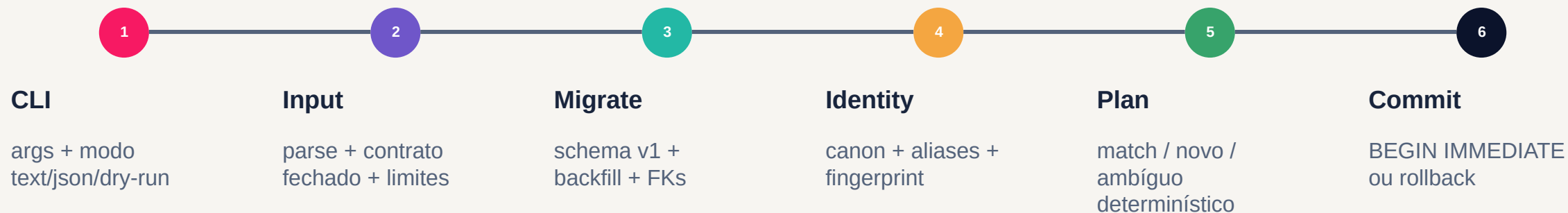
Semântica de ID

255 IDs globais, mas 268 pares seller+Id: o ID é texto opaco e escopado por seller.

Integridade da fonte

JSON: 48.938 bytes, SHA 1b0c861f... · DB: 61.440 bytes, SHA 733ff1d9...

Arquitetura do consolidator



Fronteira de produção

Node.js + TypeScript strict + node:sqlite + filesystem. Nenhuma rede, nenhum modelo e nenhum SDK de provider.

Validação fechada: rejeitar cedo, explicar sempre

Forma

array JSON; objetos com exatamente 5 campos

Tipos

strings não vazias; Brand aceita null

Limites

5 MiB · 10.000 linhas · 1.000 chars/campo

Diagnóstico

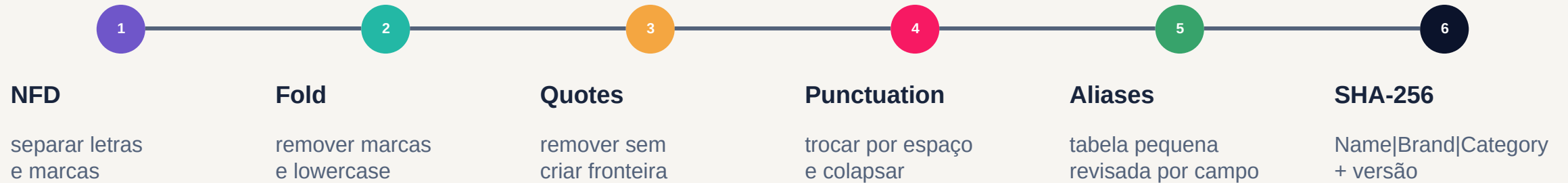
até 20 erros estáveis, sem ecoar dados sensíveis

Deduplicação

mesmo seller+Id idêntico colapsa; conflito aborta

Resultado: nenhuma linha parcialmente aceita entra na transação.

Identidade canônica v1



IDENTIDADE EXPLICÁVEL

```
fingerprint = sha256("v1\0" + canonicalName + "\0" + canonicalBrand + "\0" + canonicalCategory)
```

Preferência explícita: um falso split é menos danoso que um falso merge silencioso.

A decisão de matching nunca é implícita



Por que não fuzzy/embedding/LLM?

Porque score não é identidade. Um falso merge corrompe o catálogo; ambiguidade explícita preserva a possibilidade de revisão humana.

Migração: constraints tornam a regra verdadeira

ANTES

Product

Id · Name · Brand · Category

SellerProduct

SellerProductId numérico
sem unicidade seller-scoped



SCHEMA v1

Product

campos de exibição preservados

ProductIdentity

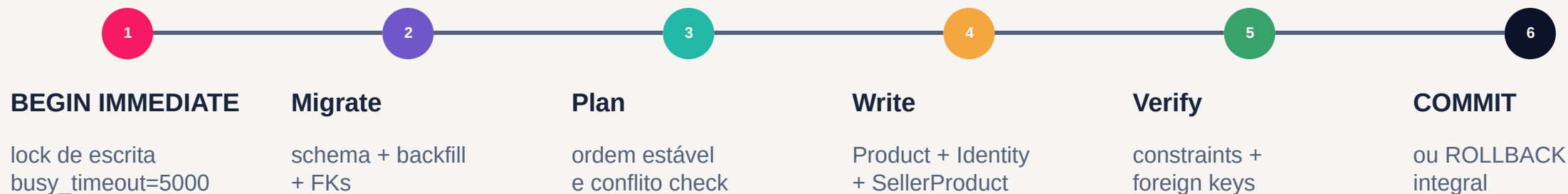
ProductId FK · version · canonical fields · UNIQUE
fingerprint

SellerProduct

SellerProductId TEXT · FK · UNIQUE(SellerName,
SellerProductId)

Backfill + collision check + foreign_key_check + PRAGMA user_version=1 — tudo transacional

Uma transação para migração + consolidação



Idempotência lógica

UNIQUE + resolução determinística: rerun não insere produto nem vínculo.

Dry run verdadeiro

Executa a mesma lógica e termina em ROLLBACK; os bytes do DB permanecem idênticos.

Contrato da CLI

USO

```
catalog-consolidate --input <products.json> --database <catalog.db> \  
[--dry-run] [--format text|json] [--debug]
```

0

sucesso no modo
solicitado

2

argumento/input
inválido

3

ambiguidade de
identidade

4

migração/
transação/DB

1

falha inesperada

Saída JSON

Envelope versionado, um único documento, sem log prose; redige o caminho do banco e inclui schemaVersion/runId/counts.

Segurança e observabilidade proporcionais

SQL injection

prepared statements; texto hostil permanece dado

Vazamento

sem raw rows, SQL, segredos ou caminhos absolutos

Disponibilidade

lock timeout previsível e erros classificados

Determinismo

runId/contagens/elapsed; nenhuma telemetria externa

Supply chain

runtime sem dependência de modelo ou SDK de provider

Resultado observado nos fixtures

1ª execução

267

matches

1

produto novo

268

links criados

2ª execução

0

novos produtos

0

novos links

268

já presentes

976

produtos finais

0

ambiguidades/rejeições

0

violações de FK

Dry-run: hash do banco antes = hash depois

Estratégia de testes: provar propriedades

30 testes públicos

CLI · validação · migração · identidade
· atomicidade · ordem invariável · lock
SQLite · segurança · erro/exit codes

4 testes de fixture

hash das fontes · 975 preservados ·
267 matches + 1 novo · 268 links ·
rerun no-op · FKs limpas · dry-run
imutável

Evidência determinística > opinião do modelo

10 specs, 3 ADRs e uma trilha pública

SDD-000 fonte/ledger ✓

SDD-001 fronteira/CLI ✓

SDD-002 input ✓

SDD-003 migração ✓

SDD-004 identidade ✓

SDD-005 consolidação ✓

SDD-006 segurança ✓

SDD-007 verificação ✓

SDD-008 entrega ✓

SDD-009 histórico/tutorial ✓

ADR-0001 autoridade · ADR-0002 identidade · ADR-0003 evidência

Como executar com as URLs oficiais

RUN SEGURO — BANCO DESCARTÁVEL

```
cd projects/catalog-consolidation
npm ci
npm run sources:ingest
cp .sdd/inputs/catalog.db /tmp/catalog-disposable.db
node src/cli.ts --input .sdd/inputs/ProductEntry.json \
  --database /tmp/catalog-disposable.db --dry-run --format json
node src/cli.ts --input .sdd/inputs/ProductEntry.json \
  --database /tmp/catalog-disposable.db --format json
```

sources:ingest baixa e verifica:

.../ProductEntry.json · .../catalog.db

VALIDAÇÃO

```
npm run check
npm run sources:ingest && npm run test:fixture
```


Alternativas rejeitadas

Fuzzy matching

Threshold arbitrário e falso merge difícil de explicar

Canon exata + ambiguidade

LLM em runtime

Variância, custo, rede e indisponibilidade no caminho crítico

Runtime model-free

Sem UNIQUE/FK

Idempotência depende do código e falha sob concorrência

Constraints SQLite

Arquitetura distribuída

Mais falhas operacionais sem requisito de escala

CLI + transação local

Commitar fixtures/PDFs

Risco de confidencialidade e histórico pesado

Hashes + inventário

O que esta solução não finge resolver

Identidade

A tupla não é universal; aliases exigem revisão e versionamento.

Escala

SQLite tem contenção bounded; horizontalização só quando dados exigirem.

Interface

É uma CLI local, não um serviço HTTP — por escolha proporcional ao desafio.

Evolução

Nova feature começa por spec/AC/ADR, orçamento, run Harness, testes e histórico.

Defesa final

É o melhor ajuste porque maximiza explicabilidade, atomicidade e evidência com a menor superfície operacional compatível com o requisito.